AI-Assisted Software Development

Custom Instructions

Mike Burton 4-6 November 2025

The Need for Custom Instructions

Consider this scenario:

- You're working on a large project
- Your team has specific coding conventions
- You use particular libraries and frameworks
- You have performance requirements

How do you ensure the AI understands all of this? 🤔

This is where custom instructions come in!

What are Custom Instructions?

Think of custom instructions like "training" your AI assistant:

- Like teaching a new team member your project's standards
- Providing context about your codebase
- Setting rules for code generation

Benefits of Custom Instructions

Let's explore why custom instructions are valuable:

1. **Customized AI Behavior**
   - Aligns with your project's specific needs
   - More relevant code suggestions
   - Fewer iterations needed
2. **Consistency Within and Across Teams**
   - Enforces coding standards automatically
   - Maintains uniform style
   - Consistent AI responses for all team members
3. **Enhanced Context Awareness**
   - AI understands your project better
   - Knows about commonly used methods
   - Aware of architectural decisions
4. **Productivity Boost**
   - Less time editing AI suggestions
   - Faster code generation
   - Reduced back-and-forth

Specifying Custom Instructions

- Specify your custom instructions in natural language
- Tools will automatically use them and send them to the AI

Global Instructions

Cursor allows you to set global instructions:

- `Cursor Settings` > `Rules` > `User Rules`

Project-Specific Instructions

Project-specific instructions let you fine-tune the AI for your project:

Cursor AI

- `Cursor Settings` > `Rules` > `Project Rules`

```
📁 your-project/
└── .cursorrules
```

GitHub Copilot

```
📁 your-project/
└── .github/
    └── copilot-instructions.md
```

Examples

Cursor Example:

```
// .cursorrules example
Prefer using async/await over Promises
Follow BEM naming convention for CSS
Use TypeScript strict mode
```

GitHub Copilot Example:

```
// copilot-instructions.md
This project:
- Uses React 18 with TypeScript
- Follows Atomic Design principles
- Requires accessibility (WCAG 2.1)
- Uses Jest for testing
```

The AI now has context for *all* interactions! 🎯

Use Cases: Coding Standards

Custom instructions can enforce:

- Naming conventions

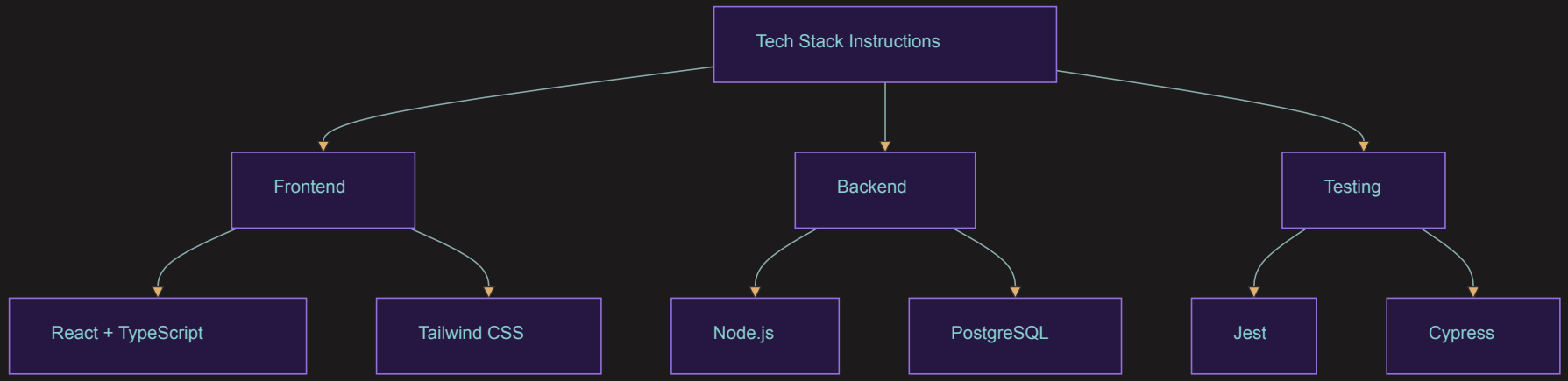
```
// ✅ Good
interface UserProfile { ... }

// ❌ Avoid
interface userprofile { ... }
```

- Code structure
- Documentation requirements
- Testing patterns

Use Cases: Technology Stack

Guide the AI about your tech choices:



Use Cases: Performance Optimization

Tell the AI to prioritize:

- ⚡ Minimizing blocking operations
- 🔄 Implementing caching strategies
- 🎯 Code splitting
- 🚀 Resource optimization

Example:

```
// AI will prefer this pattern
const heavyOperation = async () => {
    const cached = await cache.get('key');
    if (cached)
        return cached;
    const result = await computeExpensiveResult();
    await cache.set('key', result);
    return result;
};
```

Ready-Made Instructions

GitHub repository of Cursor rules:

- https://github.com/PatrickJS/awesome-cursorrules/

Other directories of Cursor rules:

- https://cursor.directory
- https://dotcursorrules.com
- https://cursorrule.com

Summary: Maximizing AI Assistant Effectiveness

Key Takeaways:

1. **Custom Instructions are Powerful**
    - Tailor AI behavior to your needs
    - Improve code quality and consistency
2. **Implementation is Simple**
    - `.cursorrules` for Cursor AI
    - `copilot-instructions.md` for GitHub Copilot
3. **Benefits are Significant**
    - Better code suggestions
    - Increased productivity
    - Improved team alignment

Remember: The better your instructions, the better your AI assistant performs! 🚀